> Question 7: Create a Jenkinsfile defining pipeline stages (build, test, deploy) and demonstrate a declarative or scripted pipeline execution.
>
> Course outcome: CO1

A declarative Jenkinsfile was written with build, test and deploy stages and executed in Jenkins. Both runs (manual and SCM-triggered) finished green.

Jenkinsfile (declarative pipeline)

```
// Declarative pipeline for the DevOps Lab 1 flask app
pipeline {
    agent any

    stages {
        stage("Checkout") {
            steps {
                echo "Pulling source code from GitHub..."
                checkout scm
            }
        }
        stage("Build") {
            steps {
                echo "Installing dependencies..."
                sh "python3 -m venv venv"
                sh "./venv/bin/pip install -r app/requirements.txt"
            }
        }
        stage("Test") {
            steps {
                echo "Running unit tests..."
                sh "./venv/bin/python -m pytest tests/ -v"
            }
        }
        stage("Package") {
            steps {
                sh "mkdir -p dist && tar czf dist/app-build-${BUILD_NUMBER}.tar.gz app tests"
                archiveArtifacts artifacts: "dist/*.tar.gz", fingerprint: true
            }
        }
        stage("Deploy") {
            steps {
                echo "Deploying app to /opt/devops-lab1 and restarting service..."
                sh "sudo bash /var/lib/jenkins/workspace/devops-lab-pipeline/scripts/deploy.sh"
            }
        }
    }

    post {
        success {
            echo "Pipeline finished: BUILD SUCCESS"
        }
        failure {
            echo "Pipeline finished: BUILD FAILED"
        }
    }
}
```

Stage description

Stage	What it does	Key step

Checkout	Pulls the repo from GitHub	checkout scm
Build	Creates a virtualenv and installs Flask	pip install -r app/requirements.txt
Test	Runs the pytest unit tests	python -m pytest tests/ -v
Package	Tars the source into dist/app-build-N.tar.gz	archiveArtifacts
Deploy	Copies app to /opt/devops-lab1, restarts systemd service, health check	scripts/deploy.sh

Execution evidence

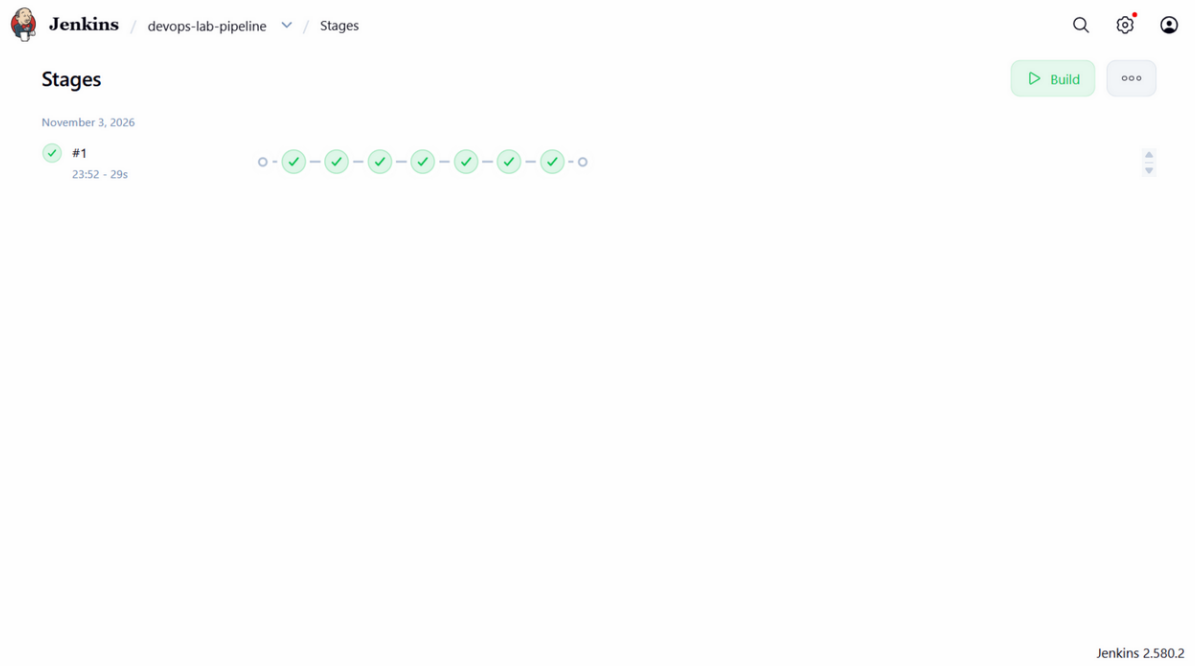


Fig: Stage view of build #1 - 7 stages, all green

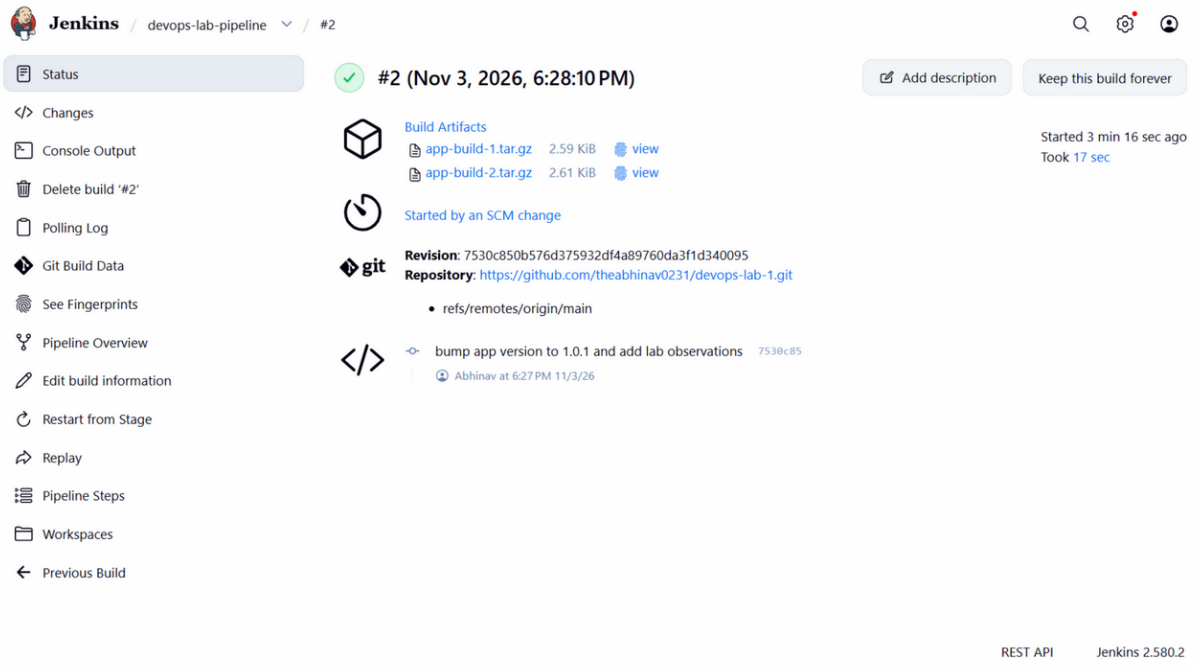


Fig: Build #2 (SCM triggered) with commit bump app version to 1.0.1

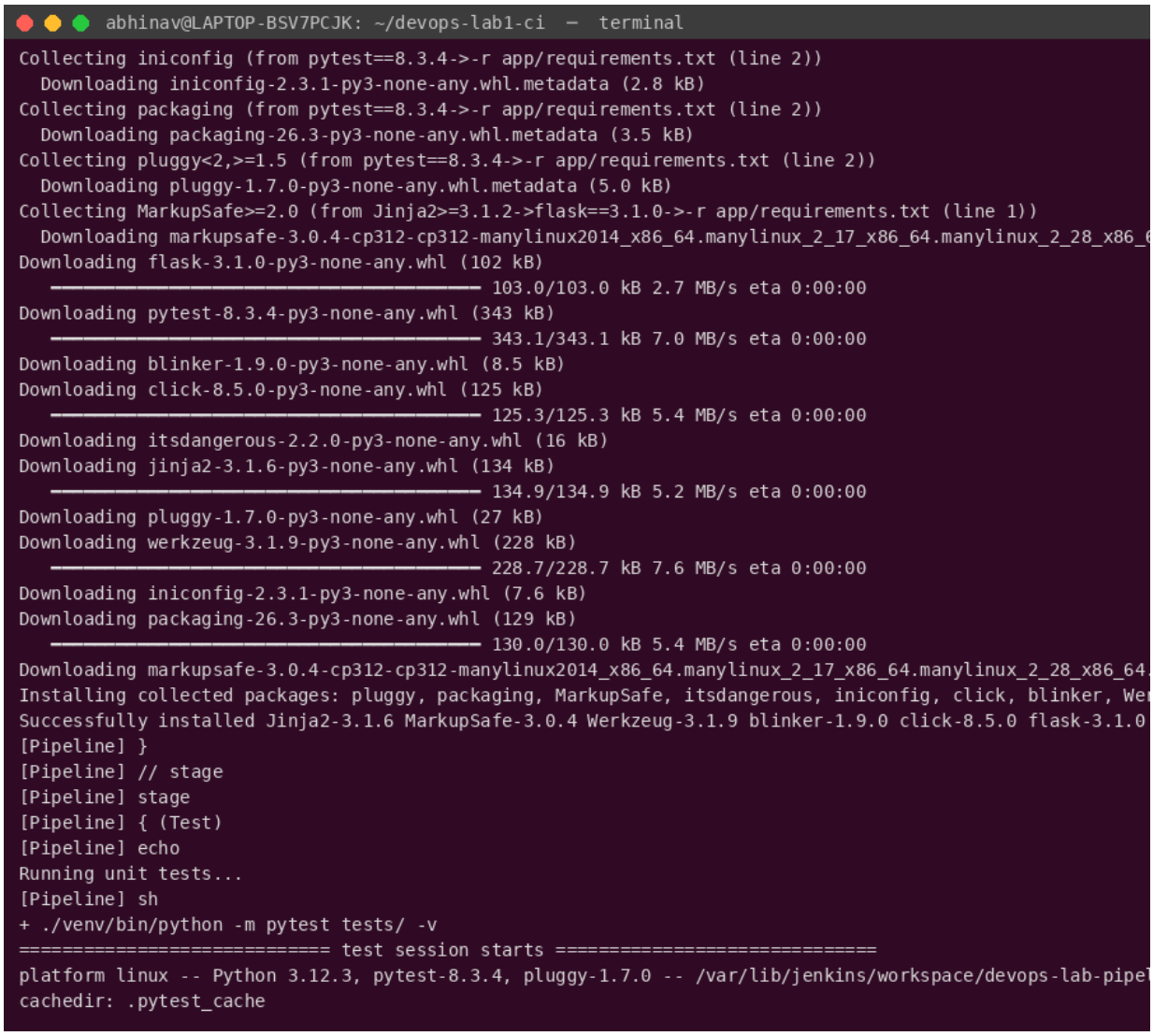


Fig: Console: package and deploy stages

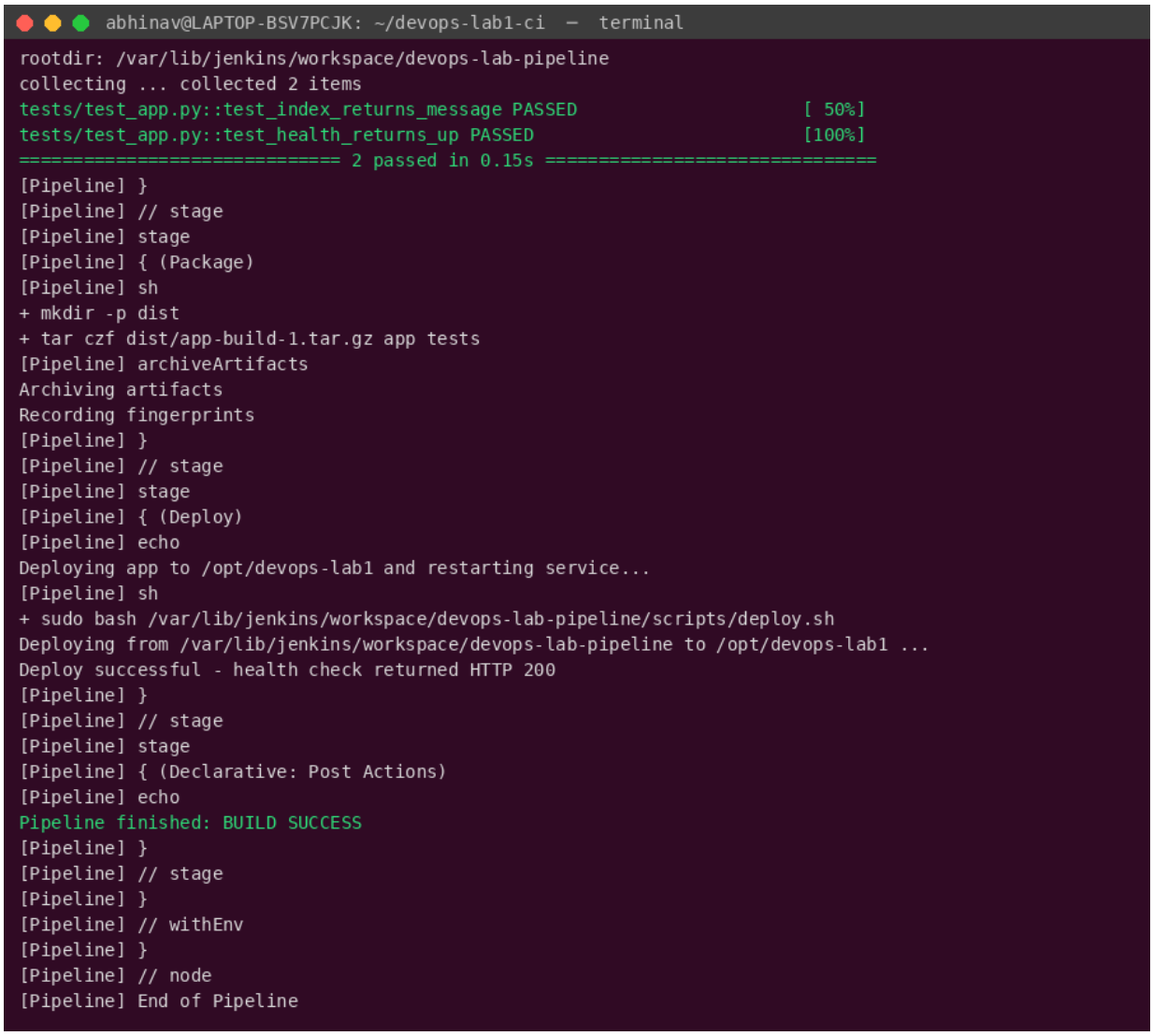


Fig: Console: post actions, Finished: SUCCESS

Observations

- Declarative syntax keeps the pipeline readable; each stage has a single responsibility.
- post { success / failure } prints a clear result line at the end of every run.
- A failing pytest or health check would stop the pipeline before the deploy stage or turn the build red, so broken code is never deployed.

Conclusion

All the commands were executed successfully on Ubuntu 24.04 LTS and the evidence (command logs, screenshots and commit history) is committed in the GitHub repository github.com/theabhinav0231/devops-lab-1 under the docs/ directory.